

PatientProfiles: An R package to identify patient characteristics in data mapped to the OMOP common data model

Martí Català¹, Mike Du¹, Yuchen Guo¹, Kim Lopez-Guell¹, Núria Mercadé-Besora¹, Xihang Chen¹, Marta Alcalde-Herraiz¹, Xintong Li¹, Daniel Prieto-Alhambra^{1,2}, and Edward Burn¹

¹ Health Data Sciences Group, Nuffield Department of Orthopaedics, Rheumatology and Musculoskeletal Sciences, University of Oxford, United Kingdom ² Department of Medical Informatics, Erasmus University Medical Center, Rotterdam, The Netherlands ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Nick Golding](#)  

Reviewers:

- [@javier-gracia-tabuenca-tuni](#)
- [@pacoramon](#)

Submitted: 09 July 2025

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)).

Summary

Real-world data (RWD) mapped to the Observational Medical Outcomes Partnership Common Data Model (OMOP CDM) offers a standardised framework for conducting observational health research across diverse data sources. However, identifying and summarising patient-level characteristics within this model often requires custom code, limiting efficiency and reproducibility. To address this, we developed the open-source PatientProfiles R package. This package streamlines the process of extracting demographic characteristics, computing intersections between cohorts and clinical events, and generating standard summaries of patient populations in OMOP CDM datasets.

Built on the tidyverse ([Wickham et al., 2019](#)) and omopgenerics infrastructure, PatientProfiles supports SQL translation for scalable database operations and includes comprehensive test coverage across multiple database systems. It provides a suite of functions grouped into demographics, intersections, summaries, utility, and mock data generation. The package is designed for transparency, modularity, and reusability in epidemiological workflows and is available via CRAN and GitHub, along with documentation and vignettes to support users.

Statement of need

Real-world data (RWD), routinely collected health data such as GP records, hospital data, and insurance claims data are valuable resources for conducting epidemiological research studies. However, with such data typically not collected primarily for research, different RWD sources can vary substantially in format and clinical coding systems. To overcome this difficulty a common data model (CDM) is often used. A CDM helps standardising data structures across various sources, enhancing data consistency, quality, and interoperability. A particularly popular data model is the Observational Medical Outcomes Partnership (OMOP) CDM, with more than 800 million patients' health care data transformed into this format ([Overhage et al., 2011](#)).

The OMOP CDM is a person-centric relational data model. Patients' data is spread across various tables related to different clinical domains with, for example, the *condition occurrence* table containing diagnoses while the *drug exposure* table contains drug prescriptions. These different clinical tables are all linked back to the *person* table which contains a unique identifier for each individual along with some key demographic data such as their date of birth.

40 Meanwhile, records in the *observation period* table define the period of calendar time over
41 which an individual is followed-up. (Blacketer, 2025)

42 One of the principal benefits of mapping data to a CDM is that it allows for the same analytic
43 code to be run across different datasets. Developing well-tested and easy-to-use software for
44 common analytic tasks can therefore bring significant benefits, both improving the speed at
45 which analyses can be performed and improving quality by reducing the amount of study-specific
46 bespoke code needing to be written.

47 Obtaining the characteristics of individuals is one of the most common first tasks when working
48 with patient-level data. In almost all analyses specific characteristics of individuals will need to
49 be identified, after which groups of individuals who share some specific common condition or
50 characteristic need to be identified and relationships between these groups are described (for
51 example the time between a given diagnosis and a health outcome of interest).

52 We created the PatientProfiles R package to support identifying patient characteristics in data
53 mapped to the OMOP CDM. It provides functionality to obtain demographic information (such
54 as age, sex, prior observation time, future observation time, and so on), describe intersections
55 between different groups of patients, and summarise the results in a standard output format.

56 State of the field

57 PatientProfiles is part of the tidyverse-oriented OMOP R ecosystem described in *Tidy R*
58 *programming with the OMOP Common Data Model* (Burn & Sabaté, 2025), alongside
59 packages such as CDMConnector, CodelistGenerator, CohortCharacteristics, CohortConstructor,
60 OmopSketch, omock, and omopgenerics. These packages are examples from a growing
61 ecosystem of OMOP R tools that supports different analytical needs. Together, they form a
62 modular stack: CDMConnector and omopgenerics support OMOP CDM references, omock
63 provides mock data, CohortConstructor creates and manipulates cohorts, PatientProfiles adds
64 demographics and intersections, and CohortCharacteristics summarises and visualises cohort
65 characteristics. The components can be combined as needed to build different analysis pipelines.

66 HADES is another established approach, providing a broad suite for end-to-end OMOP
67 analytics, including population characterisation, causal effect estimation, and patient-level
68 prediction (Schuemie et al., 2024). HADES emphasises specifying and executing large-scale
69 observational analyses and exploring their standardised results through integrated workflows
70 and user interfaces. PatientProfiles instead contributes a focused, composable layer for
71 patient characteristics, intersections, and standard summaries, providing a complementary
72 modular approach aligned with tidyverse-style R programming. Community projects such as
73 HadesExtras (FinnGen, n.d.) illustrate efforts to extend HADES workflows; the two ecosystems
74 are complementary and fit different needs rather than competing alternatives.

75 Design principles

76 PatientProfiles was designed to adhere to the tidyverse tidy design principles. The tidyverse
77 is a collection of R packages designed for data science, offering a cohesive and consistent
78 syntax for data manipulation, and analysis (Wickham et al., 2019). The dplyr package defines
79 multiple methods that can be implemented to many different sources of data. Of particular
80 relevance to working with OMOP CDM data which is typically stored in a database, the dbplyr
81 package provides translations of dplyr methods to SQL.

82 The core dependency of PatientProfiles is the omopgenerics package (Català & Burn, 2026),
83 which provides methods, classes and basic operations for packages working with data in the
84 OMOP CDM format. It defines a central object, a `cdm_reference`, that provides a central
85 reference to all the different OMOP CDM tables, along with various other S3 classes and
86 methods that facilitate working with the data contained in this reference.

Development of the PatientProfiles R package

PatientProfiles was developed in accordance with best practices for R packages with the devtools and usethis R packages used for common development tasks. The core, general dependencies of the package include dplyr and tidyr for common data manipulations and dbplyr which provides translations to SQL. In addition the core dependency related to OMOP CDM data is the omopgenerics package which provides core classes and methods specific to this data format.

The PatientProfiles package includes functionality to create its own mock data in the OMOP CDM format. This mock data is used to test the package using the testthat framework (Wickham, 2011). Every line of the packages is tested multiple times trying to account for various edge cases. Currently, the package is tested iteratively against different database management systems: PostgreSQL, SQL Server, Amazon Redshift, and DuckDB. In addition to unit tests, end-to-end integration tests of the package have been conducted to ensure the face validity of results.

The package is open-source and released via CRAN: <https://CRAN.R-project.org/package=PatientProfiles> (Català et al., 2026) (version 1.6.1, released 5 August 2026) and also available on GitHub: <https://github.com/darwin-eu/PatientProfiles> with its own website with more documentation and vignettes that cover the content of the package in more depth.

Overview of the PatientProfiles R package

PatientProfiles contains three main groups of functions (Figure 1). **Demographics** functions are used to add information contained in person and observation period tables to other tables or objects of interest. **Intersections** are used to intersect a table with an object of interest (it can be another table, a cohort of patients or a particular clinical concept). The **summarise** functions are used to create standard objects that summarise the content of a table of interest. Finally, the package also contains some complementary utility functions to, for example, create mock data.

PatientProfiles works with valid OMOP cohort tables; apart from generating mock data for examples and tests, cohort creation and modification are outside the scope of the package. Dedicated OMOP packages such as CohortConstructor, CDMConnector (Inberg et al., 2026), CohortGenerator, and other cohort-generation tools can be used to create and modify cohorts. PatientProfiles then adds characteristics, identifies intersections, and summarises the resulting cohort tables.

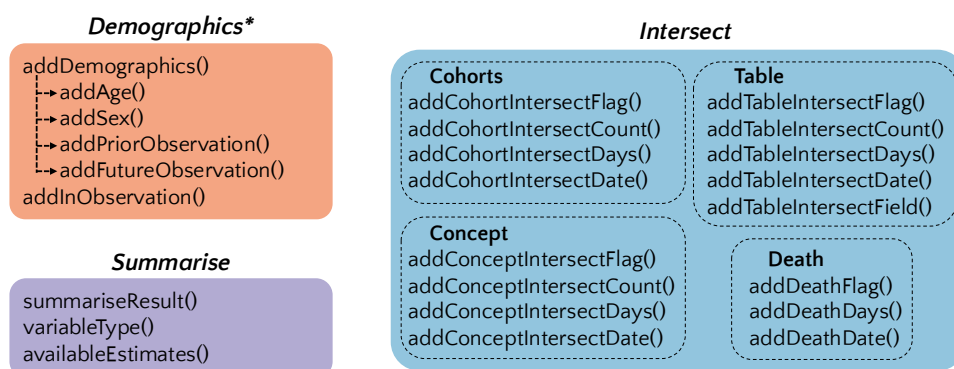


Figure 1: PatientProfiles functions bloks. Note that each demographic function has its own analogous *query* function to only add a query to the data, e.g. `addAge()` -> `addAgeQuery()`.

119 Mock data

120 A reference to an OMOP CDM instance is needed to use PatientProfiles. In this simple
 121 tutorial we will use mock toy data produced by the same package, using the omock mock-data
 122 infrastructure (Du et al., 2025), and request that it is copied into an in-process duckdb
 123 database.

```
124 library(PatientProfiles)
125 library(CohortConstructor)
126 cdm <- mockPatientProfiles(numberIndividuals = 1000, source = "duckdb")
127 # use CohortConstructor to customise cohorts
128 cdm$my_flu_cohort <- cdm$cohort1 |>
129   subsetCohorts(cohortId = 1, name = "my_flu_cohort") |>
130   renameCohort(newCohortName = "flu")
131 cdm$target <- cdm$cohort2 |>
132   renameCohort(
133     cohortId = c(1L, 2L, 3L),
134     newCohortName = c("covid_test", "flu_test", "asthma")
135   ) |>
136   dplyr::compute(name = "target")
```

137 Demographics

138 addDemographics() is used to characterise the demographics of a table. The table must be
 139 part of a cdm_reference object and contain a person identifier column (either person_id or
 140 subject_id). There are multiple columns that can be added with this function:

- 141 ▪ *age*: the age at a certain indexDate. You can also add an *age group* column grouping
 142 individuals into different age ranges.
- 143 ▪ *sex*: the sex of the individual.
- 144 ▪ *prior observation*: the number of days between start of observation and indexDate.
- 145 ▪ *future observation*: the number of days between indexDate and end of observation.
- 146 ▪ *date of birth*: the birth date of the individual.

147 An example to add the demographics to a mock cohort table is:

```
148 cdm$my_flu_cohort |>
149   addDemographics(
150     indexDate = "cohort_start_date",
151     ageGroup = list("children" = c(0, 17), "adult" = c(18, Inf))
152   ) |>
153   dplyr::glimpse()

154 ## Rows: ??
155 ## Columns: 10
156 ## Database: DuckDB v1.0.0 [root@Darwin 23.4.0:R 4.4.1/:memory:]
157 ## $ cohort_definition_id <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1...
158 ## $ subject_id          <int> 117, 818, 634, 729, 886, 245, 761, 385, 597, 53, ...
159 ## $ cohort_start_date   <date> 1959-06-03, 1936-09-03, 1979-08-13, 2010-03-
160 14, ...
161 ## $ cohort_end_date     <date> 1960-11-29, 1981-03-23, 2053-12-02, 2044-04-
162 28, ...
163 ## $ age                 <int> 27, 2, 16, 36, 58, 20, 38, 82, 94, 77, 18, 5, 5, ...
164 ## $ age_group           <chr> "adult", "children", "children", "adult", "adult"...
165 ## $ sex                 <chr> "Female", "Female", "Female", "Male", "Male", "Ma...
166 ## $ prior_observation   <int> 10015, 976, 6068, 13221, 21371, 7397, 13961, 3005...
167 ## $ future_observation  <int> 768, 41462, 32133, 40592, 20967, 10212, 20892, 19...
```

```
168 ## $ date_of_birth      <date> 1932-01-01, 1934-01-01, 1963-01-01, 1974-01-
169 01, ...
170 For each functionality, there is an individual function: addAge( ), addSex( ), addPriorObservation( ),
171 addFutureObservation( ) or addDateOfBirth( ).
```

172 Observation period id

173 The *observation_period* contains the period of time that an individual in the database is in
 174 observation. There might be multiple individual periods per person, but they can not overlap
 175 each other. When doing analysis it can be of interest knowing if a certain date is in observation,
 176 whether the individual will be in observation after a certain time, and from which observation
 177 period is an observation. To do so we have two functions:

- 178 ■ addInObservation() to identify if an individual is in observation in a certain *window*
 179 with respect to an *indexDate*.
- 180 ■ addObservationPeriodId() to identify the observation period ordinal for that date.

```
cdm$my_flu_cohort |>
  addInObservation(
    indexDate = "cohort_start_date",
    window = list("obs_index_date" = c(0, 0), "in_1_year" = c(365, 365)),
    nameStyle = "{window_name}"
  ) |>
  addObservationPeriodId() |>
  dplyr::glimpse()
```

```
181 ## Rows: ??
182 ## Columns: 7
183 ## Database: DuckDB v1.0.0 [root@Darwin 23.4.0:R 4.4.1/:memory:]
184 ## $ cohort_definition_id <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...
185 ## $ subject_id          <int> 962, 1158, 4462, 351, 3556, 320, 1965, 2105, 259...
186 ## $ cohort_start_date   <date> 1995-07-09, 2016-12-27, 1990-10-23, 2018-06-
187 28,...
188 ## $ cohort_end_date     <date> 2019-06-14, 2017-02-15, 2018-04-27, 2018-06-
189 29,...
190 ## $ obs_index_date      <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...
191 ## $ in_1_year           <int> 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, ...
192 ## $ observation_period_id <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...
```

193 Query functions

194 Usually OMOP CDM instances are stored in SQL databases. The functions that we have
 195 seen take the original table add the new columns and save the result into a new table (name
 196 argument). Each function has its own homologous one (same name terminated with 'Query')
 197 that instead of saving the result to a table only returns a query to generate the table. For
 198 local instances both functions provide exactly the same result.

199 Intersections

200 PatientProfiles has 15 functions that are used to compute intersections between tables,
 201 cohorts, and concepts. Common function parameters are:

- 202 ■ indexDate: Name of the column that contains the date that will be the origin time of
 203 our calculations.
- 204 ■ censorDate: Name of the column that contains the date to censor the observation
 205 window.

206 ▪ window: Window of time with respect to the index date that we will consider relevant
207 events.

208 There are 5 different function types:

- 209 ▪ *Flag*: Creates a new integer column that can have three possible values: 1 if an event
210 of interest is observed; 0 if the event is not observed; and NA if the individual is not in
211 observation within that window.
- 212 ▪ *Count*: Creates a new integer column with the number of observed events; NA is reported
213 if the individual is not in observation in that window.
- 214 ▪ *Date*: Creates a new date column containing the date of a certain event; NA is reported
215 if the event is not observed or the individual is not in observation in that window.
- 216 ▪ *Days*: Creates a new integer column containing the time difference to a certain event;
217 NA is reported if the event is not observed or the individual is not in observation in that
218 window.
- 219 ▪ *Field*: Creates a new column containing a selected field from the first or last matching
220 event in the window; NA is reported if no event is observed or the individual is not in
221 observation in that window.

222 For the *Flag* and *Count* functions there are two extra parameters: - targetStartDate: Name
223 of the column that identifies the start of the event. - targetEndDate: Name of the column
224 identifying the end of the episode. If NULL, the event is considered to start and end on
225 targetStartDate.

226 With the following code you can add the number of visits recorded in the prior year
227 (number_visits) and a flag to see if there is a record of asthma at any time before the index
228 date.

```
229 cdm$my_flu_cohort |>
230   addTableIntersectCount(
231     tableName = "visit_occurrence",
232     window = c(-365, 0),
233     nameStyle = "number_visits"
234   ) |>
235   addCohortIntersectFlag(
236     targetCohortTable = "target",
237     targetCohortId = 3,
238     window = c(-Inf, 0),
239     nameStyle = "prior_asthma"
240   ) |>
241   dplyr::glimpse()

242 ## Rows: ??
243 ## Columns: 6
244 ## Database: DuckDB v1.0.0 [root@Darwin 23.4.0:R 4.4.1/:memory:]
245 ## $ cohort_definition_id <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1...
246 ## $ subject_id          <int> 117, 818, 634, 886, 245, 761, 597, 53, 124, 285, ...
247 ## $ cohort_start_date   <date> 1959-06-03, 1936-09-03, 1979-08-13, 1996-07-
248   06, ...
249 ## $ cohort_end_date     <date> 1960-11-29, 1981-03-23, 2053-12-02, 2048-10-
250   18, ...
251 ## $ number_visits       <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0...
252 ## $ prior_asthma        <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0...
```

253 For the *Date*, *Days*, and *Field* functions there are two extra parameters: - targetDate: Name
254 of the column that contains the event of interest. - order: Whether we are interested in the
255 "first" or "last" event in the window.

256 The *Field* functions also require field, the name of the field to return from the selected event.

```

257 With the following code you would add the date of the next test (flu or COVID) after the
258 index date:

cdm$my_flu_cohort |>
  addCohortIntersectDate(
    targetCohortTable = "target",
    targetCohortId = c(1, 2),
    window = c(1, Inf)
  ) |>
  dplyr::glimpse()

259 ## Rows: ??
260 ## Columns: 6
261 ## Database: DuckDB v1.0.0 [root@Darwin 23.4.0:R 4.4.1:/memory:]
262 ## $ cohort_definition_id <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,...
263 ## $ subject_id <int> 761, 597, 124, 285, 157, 348, 297, 919, 830, 741,...
264 ## $ cohort_start_date <date> 2037-03-23, 2093-08-24, 1981-12-13, 1993-01-
265 28, ...
266 ## $ cohort_end_date <date> 2058-09-28, 2141-02-14, 2020-03-08, 2006-09-
267 29, ...
268 ## $ covid_test_1_to_inf <date> 2077-08-29, NA, NA, NA, 2043-06-27, NA, NA, 2045...
269 ## $ flu_test_1_to_inf <date> NA, 2194-01-24, 2058-08-07, 2093-09-06, NA, 2041...

270 NOTE that each function has some arguments related to the intersecting target (cohort,
271 concept or clinical table).

```

272 **Summarise data**

```

273 summariseResult() allows users to summarise multiple columns to multiple estimates (see
274 availableEstimates()) in a standard output format, as shown below:

275 cdm$my_flu_cohort |>
276   addDemographics() |>
277   addTableIntersectCount(
278     tableName = "visit_occurrence",
279     window = c(-365, 0),
280     nameStyle = "number_visits"
281   ) |>
282   addCohortIntersectFlag(
283     targetCohortTable = "target",
284     targetCohortId = 3,
285     window = c(-Inf, 0),
286     nameStyle = "prior_asthma"
287   ) |>
288   addCohortIntersectDate(
289     targetCohortTable = "target",
290     targetCohortId = c(1, 2),
291     window = c(1, Inf)
292   ) |>
293   addCohortName() |>
294   summariseResult(
295     group = "cohort_name",
296     strata = list("sex", c("sex", "prior_asthma")),
297     variables = list(c("number_visits", "age"), c("covid_test_1_to_inf", "flu_test_1_to_inf")),
298     estimates = list(c("median", "q25", "q75"), c("min", "max"))
299   ) |>
300   dplyr::glimpse()

```

```

301 ## Rows: 84
302 ## Columns: 13
303 ## $ result_id      <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,...
304 ## $ cdm_name       <chr> "PP MOCK", "PP MOCK", "PP MOCK", "PP MOCK", "PP MOCK"...
305 ## $ group_name     <chr> "cohort_name", "cohort_name", "cohort_name", "cohort_...
306 ## $ group_level    <chr> "flu", "flu", "flu", "flu", "flu", "flu", "flu", "flu"...
307 ## $ strata_name     <chr> "overall", "overall", "overall", "overall", "overall"...
308 ## $ strata_level   <chr> "overall", "overall", "overall", "overall", "overall"...
309 ## $ variable_name  <chr> "number records", "number subjects", "age", "age", "a...
310 ## $ variable_level <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N...
311 ## $ estimate_name  <chr> "count", "count", "median", "q25", "q75", "median", "...
312 ## $ estimate_type  <chr> "integer", "integer", "integer", "integer", "integer"...
313 ## $ estimate_value <chr> "350", "350", "49", "23", "87", "0", "0", "0", "1945-
314 ...
315 ## $ additional_name <chr> "overall", "overall", "overall", "overall", "overall"...
316 ## $ additional_level <chr> "overall", "overall", "overall", "overall", "overall"...

```

Conclusions

The PatientProfiles R package provides functionality to assist researchers working with data mapped to the OMOP CDM format. By basing the package around this data model which has a known structure the package could be developed with simple interfaces yet deep functionality. The package has already been used in published studies (Català et al., 2024; Mercadé-Besora et al., 2024) and is freely available to be used in future research.

Funding information

Development of the PatientProfiles R package was funded by the European Medicines Agency as part of the Data Analysis and Real World Interrogation Network (DARWIN EU®). This manuscript represents the views of the DARWIN EU® Coordination Centre only and cannot be interpreted as reflecting the views of the European Medicines Agency or the European Medicines Regulatory Network.

References

- Blacketer, C. (2025). *Definition and DDLs for the OMOP common data model (CDM)*.
- Burn, E., & Sabaté, M. C. (2025). *Tidy r programming with the OMOP common data model*. Zenodo. <https://doi.org/10.5281/zenodo.17532124>
- Català, M., & Burn, E. (2026). *Omopgenerics: Methods and classes for the OMOP common data model*. <https://doi.org/10.32614/CRAN.package.omopgenerics>
- Català, M., Guo, Y., Du, M., Lopez-Guell, K., Burn, E., & Mercade-Besora, N. (2026). *PatientProfiles: Identify characteristics of patients in the OMOP common data model*. <https://doi.org/10.32614/CRAN.package.PatientProfiles>
- Català, M., Mercadé-Besora, N., Kolde, R., Trinh, N. T. H., Roel, E., Burn, E., Rathod-Mistry, T., Kostka, K., Man, W. Y., Delmestri, A., Nordeng, H. M. E., Uusküla, A., Duarte-Salles, T., Prieto-Alhambra, D., & Jödicke, A. M. (2024). The effectiveness of COVID-19 vaccines to prevent long COVID symptoms: Staggered cohort study of data from the UK, Spain, and Estonia. *The Lancet Respiratory Medicine*, 12, 226–240. [https://doi.org/10.1016/S2213-2600\(23\)00414-9](https://doi.org/10.1016/S2213-2600(23)00414-9)
- Du, M., Mercadé-Besora, N., Alcalde-Herraiz, M., Chen, X., Guo, Y., López-Güell, K., Burn, E.,

- 345 & Català, M. (2025). Omock: A r package for mock data generation for the observational
346 medical outcomes partnership common data model. *Journal of Open Source Software*,
347 10(113), 8178. <https://doi.org/10.21105/joss.08178>
- 348 FinnGen. (n.d.). *HadesExtras*. Retrieved September 9, 2026, from [https://github.com/](https://github.com/FINNGEN/HadesExtras)
349 [FINNGEN/HadesExtras](https://github.com/FINNGEN/HadesExtras)
- 350 Inberg, G., Black, A., Gorbachev, A., Burn, E., Català, M., & Nika, I. (2026). *CDMConnector*:
351 *Connect to an OMOP common data model*. [https://doi.org/10.32614/CRAN.package.](https://doi.org/10.32614/CRAN.package.CDMConnector)
352 [CDMConnector](https://doi.org/10.32614/CRAN.package.CDMConnector)
- 353 Mercadé-Besora, N., Li, X., Kolde, R., Trinh, N. T., Sanchez-Santos, M. T., Man, W. Y., Roel,
354 E., Reyes, C., Delmestri, A., Nordeng, H. M. E., Uusküla, A., Duarte-Salles, T., Prats,
355 C., Prieto-Alhambra, D., Jödicke, A. M., & Català, M. (2024). The role of COVID-19
356 vaccines in preventing post-COVID-19 thromboembolic and cardiovascular complications.
357 *Heart*, 110(9), 635–643. <https://doi.org/10.1136/heartjnl-2023-323483>
- 358 Overhage, J. M., Ryan, P. B., Reich, C. G., Hartzema, A. G., & Stang, P. E. (2011). Validation
359 of a common data model for active safety surveillance research. *Journal of the American*
360 *Medical Informatics Association*, 19(1), 54–60. [https://doi.org/10.1136/amiajnl-2011-](https://doi.org/10.1136/amiajnl-2011-000376)
361 [000376](https://doi.org/10.1136/amiajnl-2011-000376)
- 362 Schuemie, M. J., Reps, J. M., Black, A., DeFalco, F., Evans, L., Fridgeirsson, E., Gilbert,
363 J. P., Knoll, C., Lavalley, M., Rao, G., Rijnbeek, P., Sadowski, K., Sena, A., Swerdel,
364 J., Williams, R. D., & Suchard, M. A. (2024). Health-analytics data to evidence suite
365 (HADES): Open-source software for observational research. *Studies in Health Technology*
366 *and Informatics*, 310, 966–970. <https://doi.org/10.3233/SHTI231108>
- 367 Wickham, H. (2011). Testthat: Get started with testing. *The R Journal*, 3, 5–10. <https://doi.org/10.32614/rj-2011-002>
- 368 [/doi.org/10.32614/rj-2011-002](https://doi.org/10.32614/rj-2011-002)
- 369 Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., Grolemond,
370 G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T. L., Miller, E., Bache, S. M.,
371 Müller, K., Ooms, J., Robinson, D., Seidel, D. P., Spinu, V., ... Yutani, H. (2019). Welcome
372 to the tidyverse. *Journal of Open Source Software*, 4(43), 1686. [https://doi.org/10.21105/](https://doi.org/10.21105/joss.01686)
373 [joss.01686](https://doi.org/10.21105/joss.01686)